

# Attention and Position Encoding

pig7selene

September 5, 2026

## Abstract

Attention is the mechanism by which a decoder-only Transformer turns a sequence of hidden states into context-dependent representations. This chapter derives Scaled Dot-Product Attention with explicit tensor shapes, explains why causal masking must precede Softmax, and relates Multi-Head Attention, Multi-Query Attention, and Grouped-Query Attention to KV Cache cost. It then develops Rotary Position Embedding as a position-dependent rotation of queries and keys whose attention score depends on relative displacement.

## 1 Introduction

The previous chapter treated causal Self-Attention as one sublayer in a decoder-only Transformer block. That abstraction is sufficient for the high-level computation graph, but it conceals the operation that gives the model access to its prefix. At every position, attention constructs a query-dependent weighted average of value vectors drawn from eligible positions. The weights are not fixed convolution coefficients or recurrent gates. They are recomputed from the current hidden states, so the same layer can retrieve a nearby syntactic cue in one context and a distant factual cue in another.

This chapter uses a batch-major hidden-state tensor  $H \in R^{B \times T \times d}$ , where  $B$  is batch size,  $T$  is sequence length, and  $d$  is model width. Let  $h$  denote the number of query heads, let  $d_h$  be the head dimension, and assume  $d = h d_h$  for the conventional full-width case. In Grouped-Query Attention,  $g$  will denote the number of key-value heads, with  $g$  dividing  $h$ . The symbols  $i$  and  $j$  index query and key positions respectively, and each ranges from 1 to  $T$  unless stated otherwise.

## 2 Scaled Dot-Product Attention

### 2.1 Query, Key, and Value Projections

Attention begins by giving each hidden state three learned roles. A query represents the information sought by a position; a key represents the kind of information a position offers; and a value is the representation that will be mixed into the output. With projection matrices  $W_Q, W_K, W_V$ , the usual batched projections are reshaped into head-major tensors,

$$Q = \text{reshape}(HW_Q) \in R^{B \times h \times T \times d_h}, \quad (1)$$

$$K = \text{reshape}(HW_K) \in R^{B \times g \times T \times d_h}, \quad V = \text{reshape}(HW_V) \in R^{B \times g \times T \times d_h}. \quad (2)$$

For ordinary Multi-Head Attention (MHA),  $g = h$ , and all three projection matrices have  $d$  output features. The more general notation in Equation 2 anticipates key-value sharing:  $W_K$  and  $W_V$  have  $g d_h$  output features when  $g < h$ . The reshaping operation changes only the view of the projected features. It does not mix token positions; all position mixing occurs in the score and weighted-sum operations below.

### 2.2 Scores, Scaling, and Weighted Values

Let  $\rho(r)$  identify the key-value head used by query head  $r$ . The unmasked score from query position  $i$  to key position  $j$  is

$$S_{b,r,i,j} = \frac{q_{b,r,i}^T k_{b,\rho(r),j}}{\sqrt{d_h}}. \quad (3)$$

Thus  $S$  has shape  $B \times h \times T \times T$ . The division by  $\sqrt{d_h}$  is the scale used in the original Transformer formulation [1]. If the components of queries and keys have comparable variance, their inner product typically grows in scale with  $d_h$ ; the normalization keeps the logits entering Softmax in a range that does not become systematically sharper solely because the head width changed.

After a mask has been added, Softmax is applied along the key-position axis:

$$A_{b,r,i,j} = \text{softmax}_j(S_{b,r,i,j} + M_{i,j}), \quad (4)$$

where  $M$  is discussed in Section 3. Each row  $A_{b,r,i,:}$  sums to one over its legal keys. The head output is then

$$O_{b,r,i} = \sum_{j=1}^T A_{b,r,i,j} v_{b,\rho(r),j} \in R^{d_h}. \quad (5)$$

The operation in Equation 5 is a content-addressed aggregation: the query selects a distribution over positions through query-key compatibility, and the selected values supply the output features. It is important that values are not themselves normalized across positions. The weights carry the positional competition; values carry the information to aggregate.

### 3 Causal Masking

A decoder-only language model must not use a future token to predict an earlier one. During training, however, it is desirable to compute all positions in parallel. A causal mask reconciles these requirements by prohibiting the score entries above the diagonal:

$$M_{i,j} = \begin{cases} 0 & j \leq i \\ -\infty & j > i. \end{cases} \quad (6)$$

Combined with Equation 4, this mask gives zero probability to every future position. The diagonal is permitted, so every query has at least its own position as a legal key. Padding or packed-sequence boundaries require additional mask terms, but they follow the same rule: inadmissible entries receive zero probability by being excluded before normalization.

The ordering is operationally significant. A correct implementation adds the mask to logits before the Softmax, often through a fused attention kernel. Setting future probabilities to zero after Softmax is not equivalent unless the remaining probabilities are renormalized. In reduced precision, implementations commonly use a representable large negative mask value or a kernel-level causal flag rather than materializing an arithmetic negative infinity. The numerical contract is nevertheless the same: masked logits contribute zero probability, and rows with no legal keys must never be normalized.

### 4 Multi-Head Attention

One attention head computes one compatibility function and one weighted mixture. MHA repeats this computation over  $h$  learned subspaces. For the conventional case with  $g = h$ , the head outputs are concatenated and mixed back into model width:

$$Y = \text{concat}(O_1, \dots, O_h) W_O, \quad Y \in R^{B \times T \times d}, \quad (7)$$

where  $W_O \in R^{d \times d}$  is the output projection. Different heads can specialize in different compatibility patterns because they use different query, key, and value projections. The heads are not independent layers: their outputs are concatenated, linearly mixed by  $W_O$ , and then added to the residual stream by the enclosing Transformer block. MHA was introduced as part of the Transformer architecture to let the model attend jointly to information from different representation subspaces [1].

The phrase “multiple heads” should not be interpreted as a guarantee of interpretable or disjoint functions. It is an architectural factorization of the attention width. Its value is that several query-key geometries can coexist at the same depth while keeping each score computation at width  $d_h$ .

## 5 Key-Value Head Sharing: MQA and GQA

During autoregressive decoding, every layer retains the keys and values of prior tokens in a KV Cache. MHA stores a distinct key and value vector for every query head. This design is expressive, but repeatedly reading that cache can become memory-bandwidth intensive as the prefix grows. Multi-Query Attention (MQA) keeps  $h$  query heads but uses a single shared key head and a single shared value head, reducing the cached key-value tensors substantially [2].

Grouped-Query Attention (GQA) interpolates between the two choices. Partition the  $h$  query heads into  $g$  contiguous groups of equal size and set

$$\rho(r) = 1 + \text{floor}\left(\frac{r-1}{\frac{h}{g}}\right). \quad (8)$$

Each group uses one key-value head. MHA is recovered by  $g = h$ , MQA by  $g = 1$ , and intermediate  $1 < g < h$  values define GQA. Ainslie et al. introduce this organization and report that it can approach MHA quality while retaining much of MQA’s decoding advantage [3]. The appropriate group count is therefore a model-and-serving trade-off, not a change to the causal attention objective.

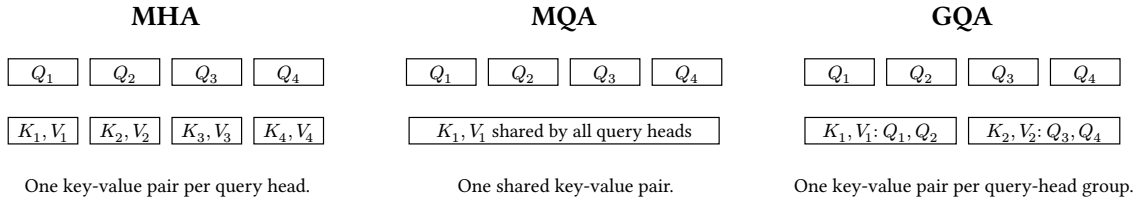


Figure 1: Key-value organization for four query heads. MHA uses four key-value heads, MQA one, and this GQA example two.

## 6 Positional Information and Rotary Position Embedding

The score in Equation 3 is permutation equivariant when the hidden states carry no positional signal: permuting the sequence permutes the projected queries, keys, values, and outputs in the same way. Token identity alone therefore cannot tell attention whether one occurrence precedes another. A Transformer needs a positional convention either in its input states or directly in its attention computation.

Rotary Position Embedding (RoPE) takes the second route. It rotates paired coordinates of each query and key by an angle determined by position before their inner product is formed. Let  $d_h$  be even and index coordinate pairs by  $m \in \{0, \dots, \frac{d_h}{2} - 1\}$ . For a frequency  $\theta_m$ , define the two-dimensional rotation

$$R_{t,m} = \begin{pmatrix} \cos(t\theta_m) & -\sin(t\theta_m) \\ \sin(t\theta_m) & \cos(t\theta_m) \end{pmatrix}. \quad (9)$$

The block-diagonal matrix  $R_t$  contains one such rotation for each coordinate pair. Standard RoPE choices use a geometric schedule such as  $\theta_m = \theta_0^{-2\frac{m}{d_h}}$ . If query  $q_t$  and key  $k_s$  occupy positions  $t$  and  $s$ , the score uses  $\hat{q}_t = R_t q_t$  and  $\hat{k}_s = R_s k_s$ . The values are not rotated for the purpose of the attention score.

### 6.1 Relative Displacement in the Attention Score

The useful algebraic property follows from orthogonality and composition of planar rotations:

$$\hat{q}_t^T \hat{k}_s = q_t^T R_t^T R_s k_s = q_t^T R_{s-t} k_s. \quad (10)$$

Although the rotation assigned to each vector uses an absolute position, the resulting query-key interaction depends on the relative displacement  $s - t$ . RoPE therefore gives the attention score an explicit relative-position structure without adding a separate position vector to the residual stream. Su et al. introduce this construction and analyze its relative-position form [4]. In practice, implementations apply the pairwise rotation after the  $Q$  and  $K$  projections and before attention-score computation; the rotation is parameter-free once the frequency schedule is fixed.

RoPE does not by itself determine a model’s usable context length or extrapolation behavior. Those properties also depend on training positions, frequency choices, scaling variants, and the rest of the architecture. Its immediate role is narrower and fundamental: it makes otherwise order-agnostic dot products sensitive to position and displacement.

## 7 Tensor Shapes, Compute, and KV Cache

### 7.1 Shape and Cache Accounting

The general  $g$ -head notation makes the principal storage effect of MQA and GQA explicit. The table records the per-layer cache in scalars for a prefix of length  $T$ . A complete model multiplies this value by its number of layers and by the byte width of its cache dtype.

| Configuration | KV heads    | Per-layer $K$ or $V$ shape       | Per-layer cache scalars |
|---------------|-------------|----------------------------------|-------------------------|
| MHA           | $g = h$     | $B \times h \times T \times d_h$ | $2BT h d_h = 2BT d$     |
| GQA           | $1 < g < h$ | $B \times g \times T \times d_h$ | $2BT g d_h$             |
| MQA           | $g = 1$     | $B \times T \times d_h$          | $2BT d_h$               |

Table 1: Key-value tensor and cache size per Transformer layer. The scalar count includes both keys and values but excludes temporary attention workspaces.

The score and probability tensors retain the query-head dimension in all three configurations:  $S, A \in R^{B \times h \times T \times T}$ . Key-value sharing changes the source tensors that those scores read, not the number of query rows. It therefore targets cache capacity and memory traffic more directly than the quadratic score shape of dense attention.

### 7.2 Compute in Training and Decoding

For a full training sequence, forming score products and weighted value sums costs on the order of  $BhT^2d_h = BT^2d$  arithmetic operations, apart from constant factors and the  $Q, K, V$ , and output projections. A naive implementation also materializes a score or probability tensor with  $BhT^2$  elements. Memory-efficient attention kernels can avoid retaining that full matrix, but they do not alter the underlying all-pairs dependency pattern.

Autoregressive decoding has a different shape. To generate one new token after a prefix of length  $T$ , each query head compares one new query with  $T$  cached keys and then combines  $T$  cached values. The attention portion is linear in prefix length for that decoding step, while the persistent cache follows Table 1. MQA and GQA reduce the number of distinct cached key-value vectors that must be read, which is why their benefit is especially relevant in bandwidth-constrained incremental serving. Projection cost, batching, kernel design, and model width still affect end-to-end latency; cache size alone is not a complete performance model.

## 8 Summary

Scaled Dot-Product Attention projects a residual stream into queries, keys, and values, turns scaled query-key inner products into a distribution over legal positions, and uses that distribution to aggregate values. Causal masking makes the parallel training computation compatible with autoregressive prediction. MHA supplies multiple learned query-key geometries; MQA and GQA preserve multiple query heads while reducing key-value cache storage and traffic. RoPE inserts position through rotations of queries and keys, giving the resulting inner product a relative-displacement form.

These mechanisms define the main sequence-mixing operator in a modern decoder-only Transformer. The following architectural topics can now treat attention as a concrete tensor program rather than a black box: normalization and residual design control the scale of its inputs, inference systems manage its cache, and distributed systems partition its projections and communication. Chapter 35 revisits MHA, MQA, and GQA as KV-state designs and separates those architectural choices from FlashAttention-style kernel execution.

## References

- [1] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.
- [2] N. Shazeer, “Fast Transformer Decoding: One Write-Head Is All You Need,” *arXiv preprint arXiv:1911.02150*, 2019, [Online]. Available: <https://arxiv.org/abs/1911.02150>
- [3] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 4895–4901. doi: 10.18653/v1/2023.emnlp-main.298.
- [4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *arXiv preprint arXiv:2104.09864*, 2021, [Online]. Available: <https://arxiv.org/abs/2104.09864>